MATRIZ DE VALIDACIÓN EN TRES CAPAS

Estrategia de Defensa en Profundidad y Entorno de Pruebas Automatizadas

Proyecto: sistema_ventas (VentasCore) **Framework:** .NET 10.0 / C# 13 **Base de Datos:** SQLite / EF Core 10 **Certificación:** 20/20 Pruebas Superadas

Capa 1: Frontend (UX & Ergonomía)
Feedback visual reactivo (`is-valid/is-invalid`), máscaras monetarias y telefónicas, y prevención de dobles envíos deshabilitando botones al interactuar.

Capa 2: Backend (Dominio & Confianza)
Validación obligatoria vía `DataAnnotations`. El servidor **no confía** en los totales enviados por el cliente: recalcula importes y verifica stock en transacciones.

Capa 3: Base de Datos (Barrera Irrevocable)
Esquema SQLite estricto con restricciones `CHECK` (precios y stocks positivos), índices únicos (evita duplicados) y llaves foráneas con `ON DELETE RESTRICT`.

MATRIZ TÉCNICA DE REGLAS DE VALIDACIÓN

MÓDULO	ID	NOMBRE DE LA REGLA	FRONTEND (COMODIDAD / UX)	BACKEND (OBLIGATORIO)	BASE DE DATOS (BARRERA FINAL)
Categorías	`VAL-CAT-01`	Nombre y longitud	Requerido, contador visual de caracteres (3 a 50).	`[Required]`, `[StringLength(50, MinimumLength=3)]`	`NOT NULL`, `VARCHAR(50)`
Categorías	`VAL-CAT-02`	Unicidad de categoría	Debounce 400ms para advertir si el nombre ya existe.	Comprobación insensible a mayúsculas (`ToLower()`).	`UNIQUE INDEX (IX_Categorias_Nombre)`
Categorías	`VAL-CAT-03`	Integridad al eliminar	Modal advirtiendo si la categoría posee productos.	Bloqueo en controlador si `Productos.Any()` (HTTP 400).	`FK (CategoriaId) ON DELETE RESTRICT`
Productos	`VAL-PROD-01`	Nombre del producto	Campo requerido, feedback visual dinámico (3-100 car.).	`[Required]`, `[StringLength(100, MinimumLength=3)]`	`NOT NULL`, `VARCHAR(100)`
Productos	`VAL-PROD-02`	Unicidad en categoría	Alerta si ya existe producto con igual nombre en la categoría.	Validación combinada: `Nombre == x && CategoriaId == y`	`UNIQUE INDEX (Nombre, CategoriaId)`
Productos	`VAL-PROD-03`	Rango de Precio Venta	Input numérico con máscara (`step="0.01"`, `min="0.01"`).	`[Range(0.01, 999999.99)]`, 2 decimales máx.	`DECIMAL(18,2)`, `CHECK(Precio > 0)`
Productos	`VAL-PROD-04`	Stock y Stock Mínimo	Inputs enteros no negativos. Alerta si Stock ≤ Mínimo.	`[Range(0, 1000000)]` para Stock y Stock_minimo.	`CHECK(Stock >= 0 AND Stock_minimo >= 0)`
Productos	`VAL-PROD-05`	Categoría asignada	Dropdown obligatorio con placeholder deshabilitado.	Comprobación de existencia de `CategoriaId` en BD.	`FOREIGN KEY REFERENCES Categorias(Id)`
Clientes	`VAL-CLI-01`	Nombre / Razón Social	Requerido, longitud permitida de 3 a 120 caracteres.	`[Required]`, `[StringLength(120, MinimumLength=3)]`	`NOT NULL`, `VARCHAR(120)`
Clientes	`VAL-CLI-02`	Formato y unicidad Email	Input `type="email"` con validación interactiva.	`[EmailAddress]`, normalización minúsculas y trim.	`UNIQUE INDEX WHERE Email IS NOT NULL`
Clientes	`VAL-CLI-03`	Formato de Teléfono	Máscara de entrada para formato telefónico nacional/int.	Regex estricto: `^\+?[0-9\s-]{7,20}$`	`VARCHAR(20) NULL`
Ventas	`VAL-VEN-01`	Cliente seleccionado	Buscador reactivo. Bloquea venta si cliente es nulo.	`ClienteId > 0` y verificación de cliente en BD.	`FK (ClienteId) ON DELETE RESTRICT`
Ventas	`VAL-VEN-02`	Canasta no vacía	Botón de "Realizar Venta" deshabilitado con carrito vacío.	Validación: `DetalleVentas.Count > 0`	Requerimiento transaccional; sin cabecera vacía.

MÓDULO	ID	NOMBRE DE LA REGLA	FRONTEND (COMODIDAD / UX)	BACKEND (OBLIGATORIO)	BASE DE DATOS (BARRERA FINAL)
Ventas	`VAL-VEN-03`	Control de Inventario	Bloqueo en UI al solicitar cantidad superior a stock.	Validación transaccional contra stock real en BD.	`CHECK(Stock >= 0)` previene sobreventa.
Ventas	`VAL-VEN-04`	Integridad de Totales	Precios en solo lectura; subtotal y total automáticos.	**Servidor ignora totales cliente**: calcula desde BD.	`DECIMAL(18,2)`, `CHECK(Total >= 0)`
Ventas	`VAL-VEN-05`	Pago y Cambio	Bloqueo si `Recibido < Total`. Muestra cambio exacto.	`Recibido >= Total` y `Cambio = Recibido - Total`	`CHECK(Recibido >= Total AND Cambio >= 0)`
Detalle Venta	`VAL-DET-01`	Cantidad por ítem	Controles incrementales `+` / `-`, mínimo 1 unidad.	`[Range(1, 100000)]`, producto existente y activo.	`INTEGER NOT NULL`, `CHECK(Cantidad > 0)`
Detalle Venta	`VAL-DET-02`	Cálculo de Subtotal	Actualización reactiva en fila del carrito.	Recálculo mandatorio en backend: `Precio * Cantidad`	`DECIMAL(18,2)`, `CHECK(Subtotal > 0)`

Entorno de Pruebas Automatizadas: `tests/sistema_ventas.Tests` con xUnit 2.9 + FluentAssertions 7.2

✓ 20 de 20 Pruebas Unitarias y de BD Superadas (100% Cobertura de Reglas)